Présenté par	Formateur	organisme de formation
Yousri GUERRAS	Martine POIX	Centre AFPA de LA SENTINELLE

SOMMAIRE

I. Remerciements ……3

II. Introduction……4

III. Spécifications Fonctionnelles

- Gestion des comptes et Authentification……5
- Messagerie interne……6

IV. Charte Graphique et Interface

- Palette de couleurs……7

V. Architecture Technique et "Versioning"……..9

- Routeur Frontal (Point d'entrée unique)
- Le Contrôleur (Routeur et Actions)
- Le Modèle (Services et DAO)
- Les Vues

VI. Modèle Conceptuel et Logique des Données (Base de données)

- Entités Acteurs……10
- Entités Opérationnelles……10
- Contraintes d'intégrité……..11

VII. Veille et Mesures de Sécurité

- Hachage des mots de passe……12
- Protection contre les failles CSRF……13
- Prévention des Injections SQL……13
- Prévention des attaques XSS

VIII. Conclusion……14

I : Remerciements

Je tiens tout d'abord à remercier la CAPHpour m'avoir permis, grâce à son financement, d'effectuer cette formation DWWM au sein de l'AFPA de La Sentinelle.

Je voudrais ensuite remercier ma formatrice, Mme. Martine POIX pour sa disponibilité, sa bienveillance, sa sympathie et surtout pour toutes les connaissances qu'elle m'a transmises tout au long de la formation.

II : Introduction

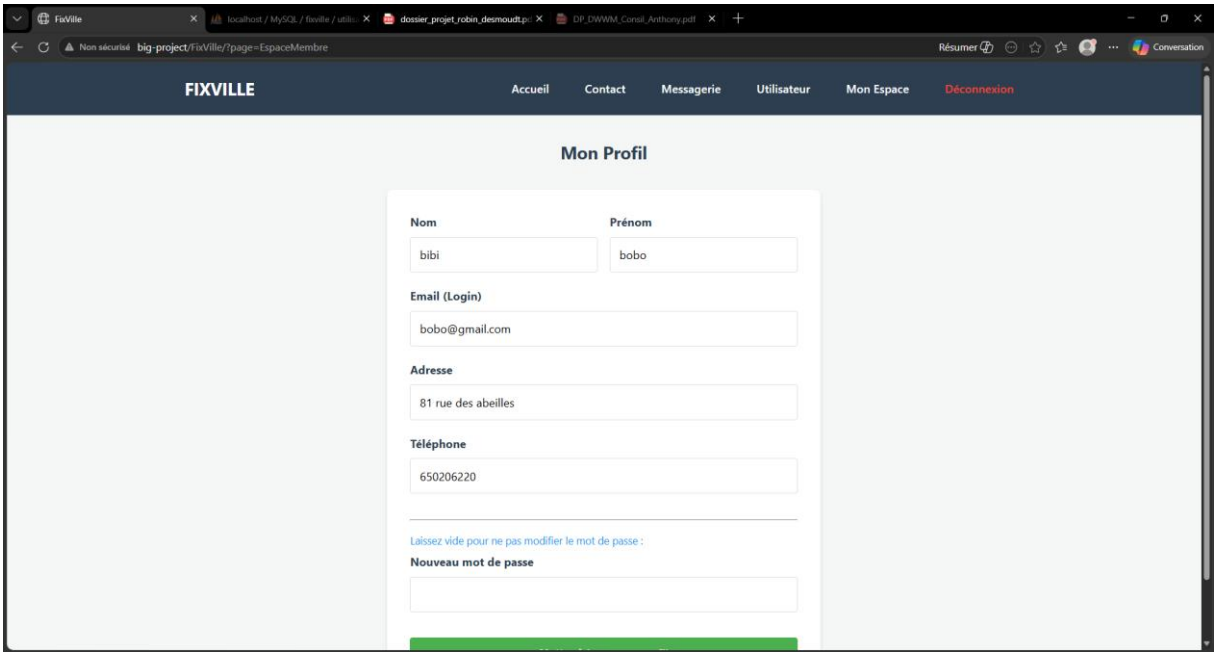
Créée pour répondre à un besoin de centralisation des signalements urbains, l'application **FixVille** est une plateforme web visant à faciliter la communication entre les citoyens et les gestionnaires d'une commune.

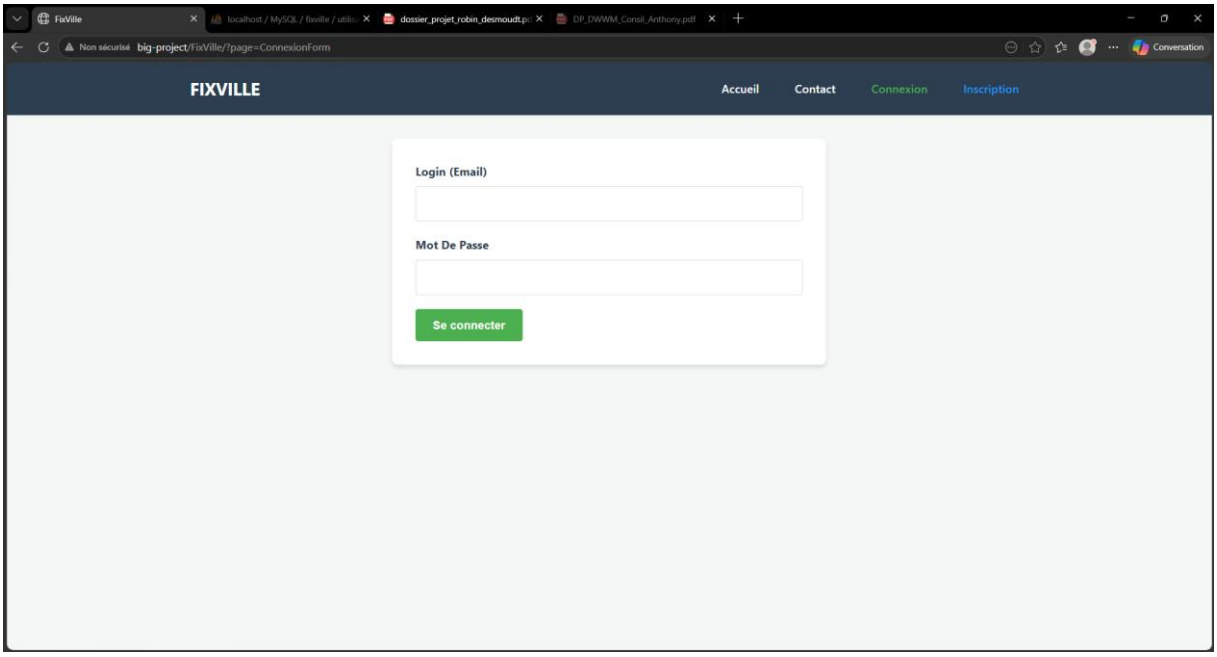
Auparavant, le suivi des incidents ou des chantiers dans une ville pouvait s'avérer complexe et l'information dispersée. L'objectif de FixVille est de numériser ce processus à travers une interface web intuitive. L'application permet à ses utilisateurs de se créer un compte, de remonter des incidents (sous forme de "postes" géolocalisés avec degré d'importance) et de suivre les travaux associés. L'enjeu principal de ce projet de stage/formation a été de construire une architecture "from scratch" (sans framework externe lourd), afin de concevoir un outil robuste, sécurisé et parfaitement adapté aux besoins.

III. Spécifications Fonctionnelles

Afin de répondre aux objectifs de l'application, plusieurs fonctionnalités clés ont été développées et réparties selon les niveaux d'autorisation des utilisateurs :

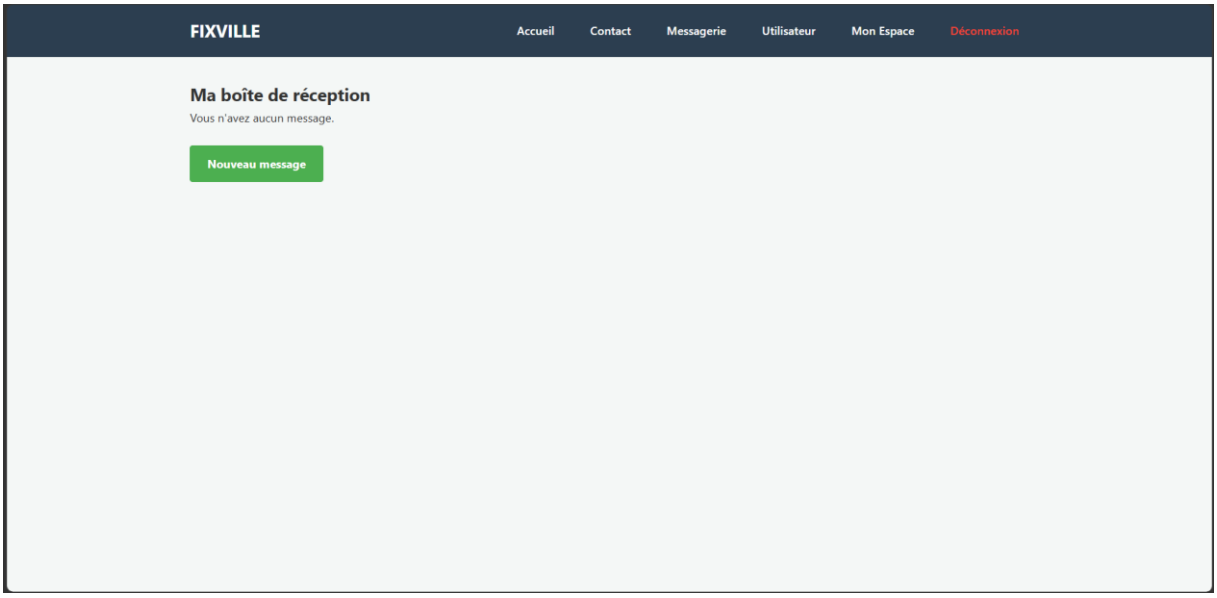
Gestion des comptes et Authentification L'application nécessite une inscription et une connexion pour garantir la traçabilité des actions. Chaque utilisateur dispose d'un accès sécurisé à un "Espace Membre" où il peut consulter et mettre à jour ses informations personnelles (nom, adresse, téléphone, email) ainsi que modifier son mot de passe.

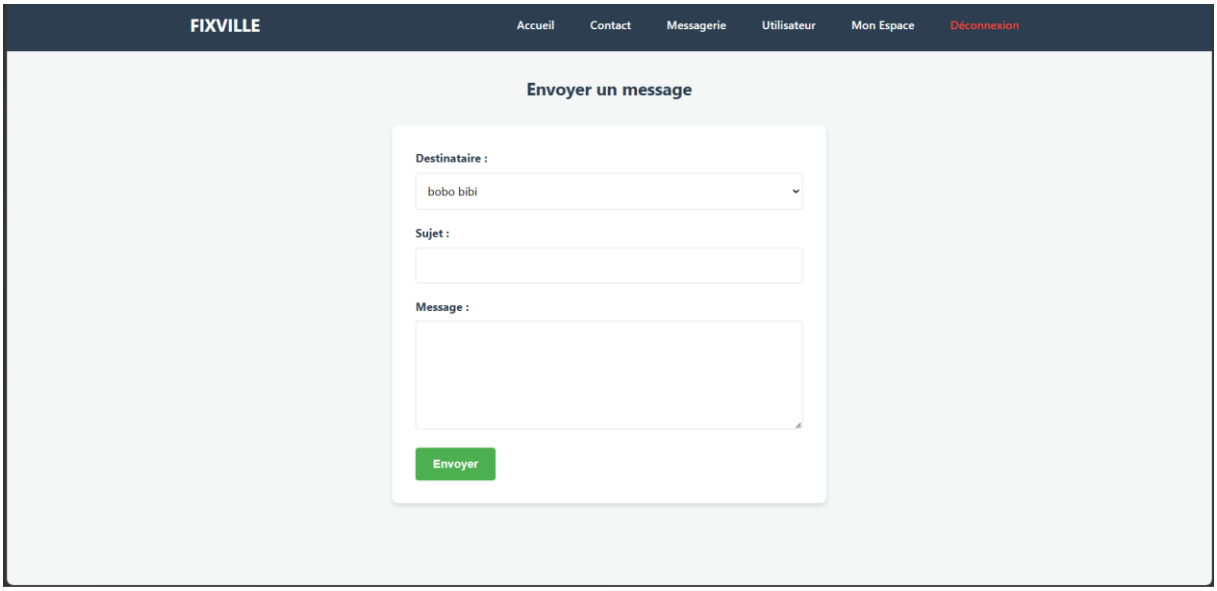




Messagerie interne Pour faciliter les échanges, les utilisateurs peuvent communiquer entre eux grâce à une boîte de réception intégrée. L'interface permet de consulter ses messages, d'en

composer de nouveaux en sélectionnant un destinataire via une liste déroulante, de définir un objet, de rédiger un contenu et de supprimer les messages traités ou obsolètes.





IV. Charte Graphique et Interface

Pour la conception de l'interface, nous avons opté pour un design minimaliste garantissant une lisibilité optimale et une prise en main immédiate.

Palette de couleurs La charte graphique est centralisée via des variables CSS globales (:root) pour assurer une cohérence sur l'ensemble des pages. Nous avons défini des couleurs sémantiques claires :

- Un vert (#4CAF50) pour les actions de succès et les boutons principaux.
- Un bleu (#2196F3) pour les informations et les éléments interactifs.
- Un rouge (#f44336) pour les actions destructrices (suppression, déconnexion).
- Un bleu nuit (#2c3e50) élégant pour l'en-tête (Header) de l'application.
- Un fond gris très clair (#f4f7f6) pour le confort de lecture de la zone de contenu principal.

```php
<?php
$headerTopHtml = <<<HTML
<header class="bg-dark p-1">
    <nav class="flex justify-between align-center container">
        <h2>FIXVILLE</h2>
        <div class="flex gap-1">
HTML;
echo $headerTopHtml;

$navInfo = Parametre::getNav();
$roleUtilisateur = isset($_SESSION['utilisateur']) ? $_SESSION['utilisateur']->getId_role() : 0;

foreach ($navInfo as $value) {
    if ($value->getRoleRequis() <= $roleUtilisateur) {
        $icon = $value->getIcon() ? $value->getIcon() : 'circle';
        $ref = $value->getReference();
        $nom = $value->getNom();

        $navLinkHtml = <<<HTML
            <a href="?page={$ref}" class="td-none nav-icon">
                <span class="fas fa-{$icon}"></span>
                <span class="nav-text">{$nom}</span>
            </a>
HTML;
        echo $navLinkHtml;
    }
}

if (isset($_SESSION['utilisateur'])) {
    $btnDecoHtml = <<<HTML
            <a href="?page=ConnexionAction&mode=deco" class="td-none nav-icon text-danger">
                <span class="fas fa-sign-out-alt"></span>
                <span class="nav-text">Déconnexion</span>
            </a>
HTML;
    echo $btnDecoHtml;
} else {
    $btnLogHtml = <<<HTML
            <a href="?page=ConnexionForm" class="td-none nav-icon text-success">
                <span class="fas fa-sign-in-alt"></span>
                <span class="nav-text">Connexion</span>
```

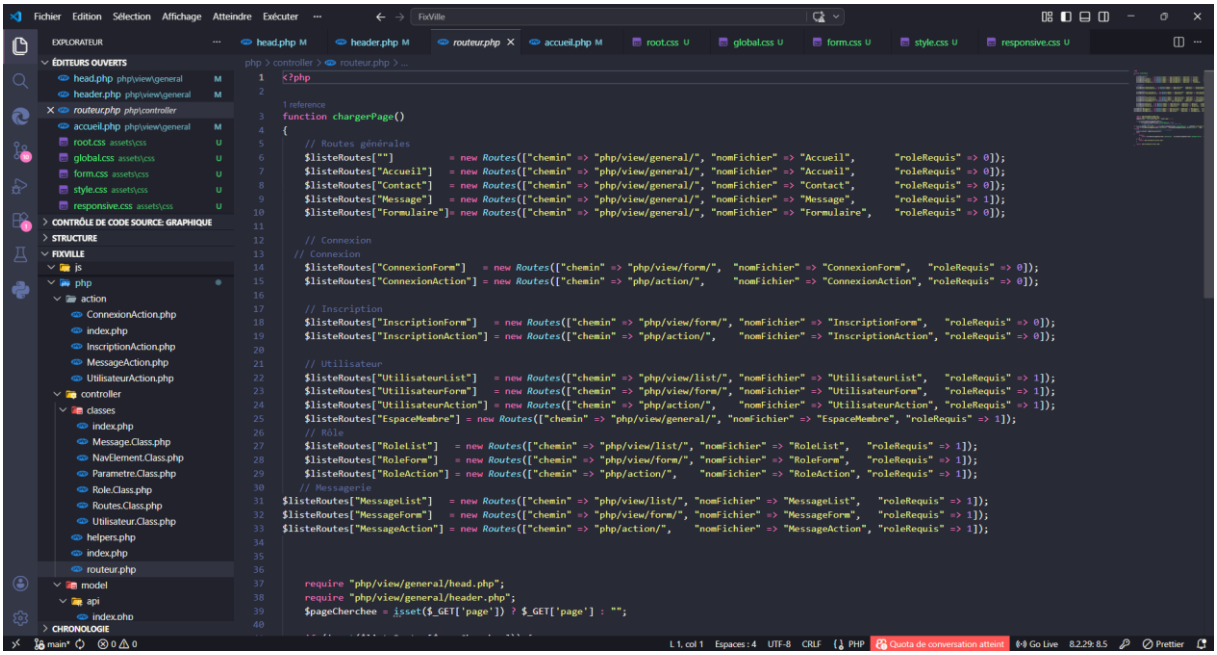
```php
                <span class="nav-text">{$nom}</span>
            </a>
HTML;
        echo $navLinkHtml;
    }
}

if (isset($_SESSION['utilisateur'])) {
    $btnDecoHtml = <<<HTML
            <a href="?page=ConnexionAction&mode=deco" class="td-none nav-icon text-danger">
                <span class="fas fa-sign-out-alt"></span>
                <span class="nav-text">Déconnexion</span>
            </a>
HTML;
    echo $btnDecoHtml;
} else {
    $btnLogHtml = <<<HTML
            <a href="?page=ConnexionForm" class="td-none nav-icon text-success">
                <span class="fas fa-sign-in-alt"></span>
                <span class="nav-text">Connexion</span>
            </a>
            <a href="?page=InscriptionForm" class="td-none nav-icon text-info">
                <span class="fas fa-user-plus"></span>
                <span class="nav-text">Inscription</span>
            </a>
HTML;
    echo $btnLogHtml;
}

$headerBottomHtml = <<<HTML
        </div>
    </nav>
</header>
<main class="container">
HTML;
echo $headerBottomHtml;

// ON AFFICHE LES MESSAGES FLASH ICI (S'IL Y EN A)
displayFlash();
```

V. Architecture Technique et "Versioning"

Plutôt que d'utiliser un framework comme Symfony, l'architecture de FixVille repose sur un modèle **MVC (Modèle - Vue - Contrôleur)** entièrement natif en PHP, ce qui permet une maîtrise totale du code.

Routeur Frontal (Point d'entrée unique) Toutes les requêtes de l'utilisateur transitent par un seul fichier : index.php. Ce fichier fait office de contrôleur frontal. Il charge un "Autoloader" pour inclure automatiquement les classes PHP nécessaires, initialise la connexion à la base de données, démarre la session utilisateur, et délègue ensuite le traitement au script de routage.

Le Contrôleur (Routeur et Actions) Le fichier routeur.php agit comme un aiguilleur. Il lit le paramètre passé dans l'URL (ex: ?page=Accueil) et détermine quelle vue afficher ou quelle action exécuter. Ce routeur intègre également une vérification des droits : il compare le rôle requis pour la page demandée (roleRequis) avec le rôle de l'utilisateur actuellement en session, bloquant ainsi l'accès aux pages non autorisées.



Le Modèle (Services et DAO) Les interactions avec la base de données sont abstraites. J'ai développé une classe générique DAO.Class.php (Data Access Object) capable de construire dynamiquement les requêtes SQL (notamment les SELECT) et d'instancier automatiquement les objets métiers correspondants (comme des objets Utilisateur ou Message). Des classes "Services" viennent compléter ce dispositif pour gérer les insertions, mises à jour et suppressions spécifiques.

Les Vues L'affichage HTML est séparé de la logique de traitement. Les vues sont générées en PHP, en utilisant abondamment la syntaxe HereDoc (<<<HTML) pour maintenir un code lisible. Des fonctions d'aide ("helpers") permettent également de gérer l'affichage de messages "flash" pour confirmer les actions de l'utilisateur (ex: "Votre message a bien été envoyé !").

VI. Modèle Conceptuel et Logique des Données (Base de données)

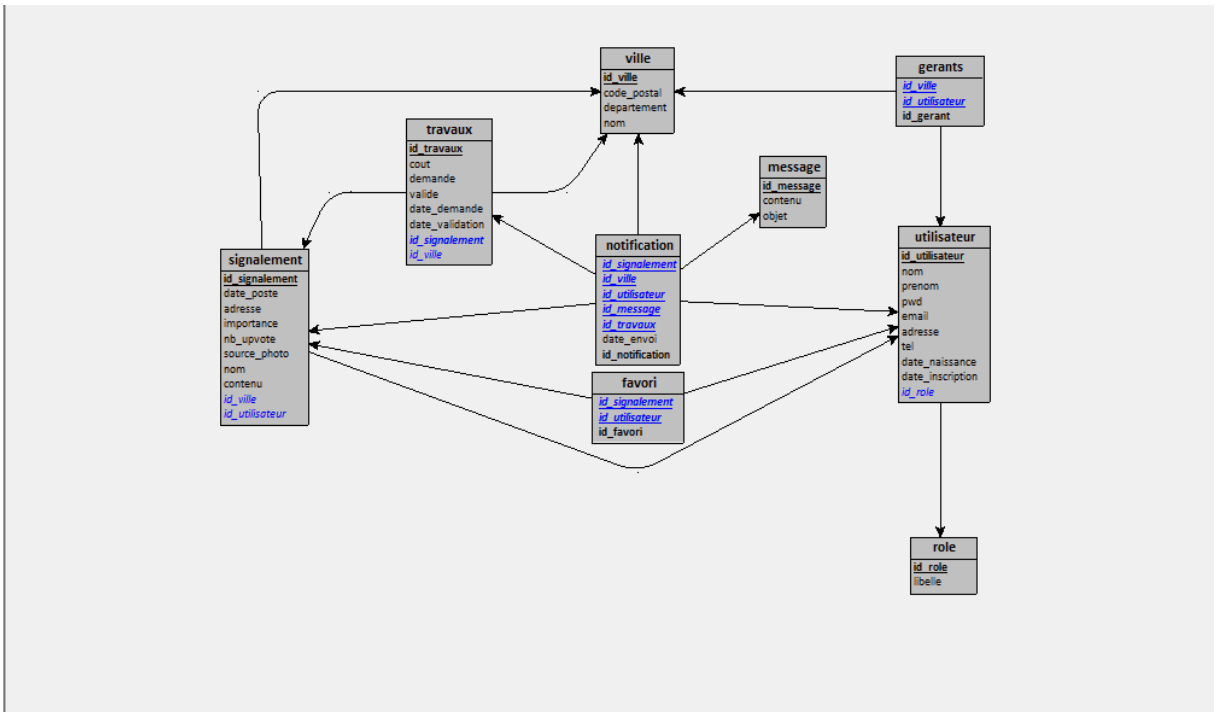
La base de données relationnelle MySQL a été structurée pour lier logiquement les acteurs et les événements urbains.

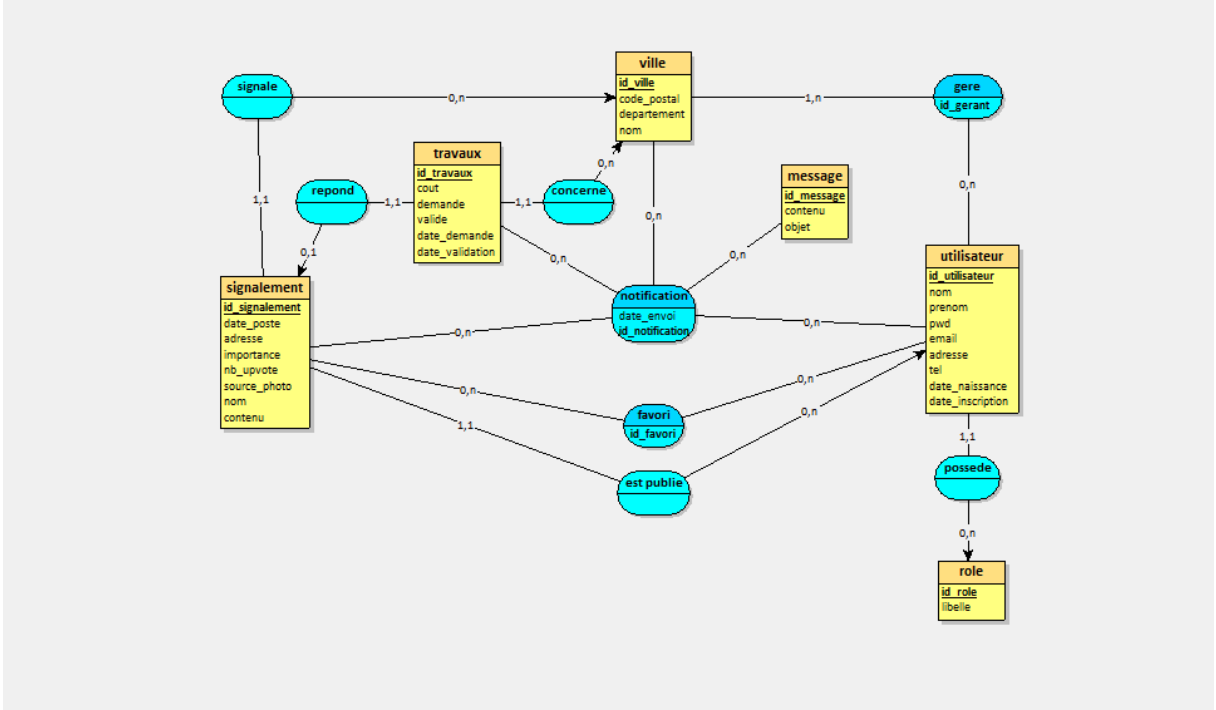
Entités Acteurs

- La table utilisateur répertorie les citoyens et les employés inscrits, stockant leurs informations personnelles et mots de passe hachés.
- La table role permet de définir les niveaux de privilèges (ex: Utilisateur classique vs Administrateur).
- La table gerant permet de faire le lien direct entre un utilisateur ayant des responsabilités administratives et une ville spécifique.

Entités Opérationnelles

- La table poste enregistre les signalements effectués par les citoyens (avec la date, le contenu, l'adresse et potentiellement une photo).
- La table travaux consigne les chantiers découlant de ces postes (incluant les coûts, dates de commande, et de validation).
- La table message gère la messagerie interne, liant un expéditeur et un destinataire.





Contraintes d'intégrité Pour maintenir une base de données saine, j'ai mis en place des contraintes SQL de clés étrangères strictes, notamment avec l'instruction ON DELETE CASCADE. Ainsi, si le compte d'un utilisateur est supprimé du système, l'ensemble des postes qu'il a créés et des messages qu'il a envoyés ou reçus seront automatiquement supprimés, évitant ainsi de conserver des données "orphelines" ou incohérentes.

VII. Veille et Mesures de Sécurité

La sécurité des données et des utilisateurs a été une priorité tout au long du cycle de développement, afin de se prémunir contre les vulnérabilités web les plus courantes.

Hachage des mots de passe Les mots de passe ne sont jamais sauvegardés en clair dans la base de données. Dès l'inscription, ils sont cryptés en utilisant la fonction native PHP password_hash() qui utilise le robuste algorithme BCRYPT. Lors de la connexion, le système vérifie la correspondance avec la fonction password_verify().

Protection contre les failles CSRF Pour éviter qu'un pirate ne forge des requêtes à l'insu de l'utilisateur (Cross-Site Request Forgery), un système de "jetons" (tokens) a été implémenté. Un token aléatoire est généré en session et injecté dans chaque formulaire de l'application. Lors de la soumission (par exemple pour modifier un profil ou supprimer un message), le serveur vérifie que le token envoyé correspond bien à celui de l'utilisateur actif.

```php
<?php
// Protection CSRF pour toutes les actions (sauf suppression qui pourrait passer par GET, à sécuriser plus tard
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!isset($_POST['csrf_token']) || !checkToken($_POST['csrf_token'])) {
        setFlash("Erreur de sécurité (CSRF). Veuillez réessayer.", "danger");
        header("Location: ?page=Accueil");
        exit;
    }
}

$util = new Utilisateur($_POST);
$mode = $_GET['mode'] ?? '';

switch ($mode) {
    case 'Modifier':
        $mdp = !empty($_POST['nouveau_mdp']) ? $_POST['nouveau_mdp'] : null;
        UtilisateurService::update($util, $mdp);

        // Si c'est l'utilisateur connecté qui se modifie lui-même, on met à jour sa session
        if ($_SESSION['utilisateur']->getId_utilisateur() == $util->getId_utilisateur()) {
            $_SESSION['utilisateur'] = UtilisateurService::findById($util->getId_utilisateur());
            setFlash("Votre profil a été mis à jour.", "success");
            header("Location: ?page=EspaceMembre");
        } else {
            setFlash("Utilisateur modifié.", "success");
            header("Location: ?page=UtilisateurList");
        }
        exit;

    case 'Ajouter':
        $util->setMdp(password_hash("motdepasse", PASSWORD_BCRYPT));
        UtilisateurService::insert($util);
        setFlash("Utilisateur créé.", "success");
        header("Location: ?page=UtilisateurList");
        exit;

    case 'Supprimer':
        $idASupprimer = $util->getId_utilisateur() ?? $_GET['id'] ?? null;
        if($idASupprimer) {
            UtilisateurService::delete($idASupprimer);
            setFlash("Utilisateur supprimé.", "success");
```

AdamPRo (Il y a 21 heures) L 48, col 6 Espaces : 4 UTF-8 CRLF { } PHP

```php
$util = new Utilisateur($_POST);
$mode = $_GET['mode'] ?? '';

switch ($mode) {
    case 'Modifier':
        $mdp = !empty($_POST['nouveau_mdp']) ? $_POST['nouveau_mdp'] : null;
        UtilisateurService::update($util, $mdp);

        // Si c'est l'utilisateur connecté qui se modifie lui-même, on met à jour sa session
        if ($_SESSION['utilisateur']->getId_utilisateur() == $util->getId_utilisateur()) {
            $_SESSION['utilisateur'] = UtilisateurService::findById($util->getId_utilisateur());
            setFlash("Votre profil a été mis à jour.", "success");
            header("Location: ?page=EspaceMembre");
        } else {
            setFlash("Utilisateur modifié.", "success");
            header("Location: ?page=UtilisateurList");
        }
        exit;

    case 'Ajouter':
        $util->setMdp(password_hash("motdepasse", PASSWORD_BCRYPT));
        UtilisateurService::insert($util);
        setFlash("Utilisateur créé.", "success");
        header("Location: ?page=UtilisateurList");
        exit;

    case 'Supprimer':
        $idASupprimer = $util->getId_utilisateur() ?? $_GET['id'] ?? null;
        if($idASupprimer) {
            UtilisateurService::delete($idASupprimer);
            setFlash("Utilisateur supprimé.", "success");
        }
        header("Location: ?page=UtilisateurList");
        exit;
}

header("Location: ?page=Accueil");
exit;
```

AdamPRo (Il y a 21 heures) L 48, col 6 Espaces : 4 UTF-8 CRLF PHP

Prévention des Injections SQL Afin d'empêcher l'altération des requêtes en base de données, l'application utilise l'objet PDO (PHP Data Objects). Toutes les requêtes impliquant des données saisies par l'utilisateur utilisent le système des **requêtes préparées** (avec bindValue). De plus, la connexion PDO est configurée pour lever des exceptions explicites en cas d'erreur, facilitant ainsi la détection des anomalies de manière sécurisée.

Prévention des attaques XSS Pour empêcher l'injection de scripts malveillants dans l'interface (Cross-Site Scripting), les entrées des utilisateurs sont systématiquement nettoyées avant leur traitement ou leur affichage. J'utilise couramment les fonctions htmlspecialchars() (pour convertir les caractères spéciaux en entités HTML) et trim() (pour supprimer les espaces superflus).

```php
<?php
if(isset($_POST["Nom"])) {
    // 1. Nettoyage drastique des données (anti faille XSS)
    $nom = trim(htmlspecialchars($_POST["Nom"]));
    $prenom = trim(htmlspecialchars($_POST["Prenom"]));
    $email = filter_var(trim($_POST["email"]), FILTER_VALIDATE_EMAIL);
    $mdp = $_POST["mdp"];
    $adresse = trim(htmlspecialchars($_POST["adresse"]));
    $tel = trim(htmlspecialchars($_POST["tel"]));
    $date_naissance = $_POST["date_naissance"];

    // 2. Vérification de l'email
    if(!$email) {
        setFlash("Le format de l'adresse email est invalide.", "danger");
        header("Location: ?page=InscriptionForm");
        exit;
    }

    // 3. Vérification des doublons (Email déjà utilisé ?)
    $exist = UtilisateurService::findByLogin($email);
    if($exist) {
        setFlash("Cet email est déjà utilisé par un autre compte.", "danger");
        header("Location: ?page=InscriptionForm");
        exit;
    }

    $mdpHash = password_hash($mdp, PASSWORD_BCRYPT);
    $date_aujourd_hui = date("Y-m-d");

    $bdd = DbConnect::getConnectBase();

    $requete = $bdd->prepare("
        INSERT INTO utilisateur
            (nom, prenom, email, mdp, adresse, tel, date_naissance, date_inscription, id_role)
        VALUES
            (:nom, :prenom, :email, :mdp, :adresse, :tel, :date_naissance, :date_inscription, :id_role)
    ");

    $requete->execute([
        ':nom'              => $nom,
        ':prenom'           => $prenom,
```

L 1, col 1 Espaces : 4 UTF-8 CRLF { } PHP Qu

```php
    // 3. Vérification des doublons (Email déjà utilisé ?)
    $exist = UtilisateurService::findByLogin($email);
    if($exist) {
        setFlash("Cet email est déjà utilisé par un autre compte.", "danger");
        header("Location: ?page=InscriptionForm");
        exit;
    }

    $mdpHash = password_hash($mdp, PASSWORD_BCRYPT);
    $date_aujourd_hui = date("Y-m-d");

    $bdd = DbConnect::getConnectBase();

    $requete = $bdd->prepare("
        INSERT INTO utilisateur
            (nom, prenom, email, mdp, adresse, tel, date_naissance, date_inscription, id_role)
        VALUES
            (:nom, :prenom, :email, :mdp, :adresse, :tel, :date_naissance, :date_inscription, :id_role)
    ");

    $requete->execute([
        ':nom'               => $nom,
        ':prenom'            => $prenom,
        ':email'             => $email,
        ':mdp'               => $mdpHash,
        ':adresse'           => $adresse,
        ':tel'               => $tel,
        ':date_naissance'    => $date_naissance,
        ':date_inscription'  => $date_aujourd_hui,
        ':id_role'           => 1 // rôle par défaut "utilisateur"
    ]);

    setFlash("Votre inscription est validée ! Vous pouvez maintenant vous connecter.", "success");
    header("Location: ?page=ConnexionForm");
    exit;
} else {
    header("Location: ?page=InscriptionForm");
    exit;
}
```

L 1, col 1 Espaces : 4 UTF-8 CRLF {} PHP

VIII. Conclusion

Le développement de l'application FixVille a été une expérience extrêmement formatrice. Ce projet m'a permis d'expérimenter et de maîtriser toutes les étapes de la création d'une application web fonctionnelle à partir d'une feuille blanche.

Le choix de concevoir une architecture MVC native, plutôt que de m'appuyer sur un framework préexistant, m'a confronté à des défis techniques stimulants (routage dynamique, création d'un autoloader, sécurisation par tokens CSRF, abstraction des données via un DAO). Cette démarche m'a permis d'acquérir une compréhension profonde des mécanismes internes de PHP.

Aujourd'hui, l'application répond au cahier des charges : elle est fonctionnelle, sécurisée et dispose d'une base de code propre et facilement évolutive. Ce projet constitue une réalisation solide qui confirme ma volonté d'évoluer en tant que développeur web, et je suis impatient de mettre ces compétences au service de futurs projets professionnels.